

Rapport de Projet : Simulation Écosystème "The Circle of Life"

1. Introduction

Dans ce projet, nous avons mis en place une simulation interactive d'un écosystème dynamique. Trois entités y évoluent : des prédateurs, des proies, et l'herbe..

Chaque individu est régi par son niveau d'énergie qui diminue au fil du temps. Pour survivre, les espèces se trouvent entre deux états : un mode passif (repos) où elles économisent leurs ressources, et un mode actif déclenché par la faim. Les proies ne consomment de l'herbe que lorsqu'elles sont actives, et elles peuvent être chassées par les prédateurs uniquement quand elles sont en train de manger. De plus, l'environnement peut subir des sécheresses ce qui rend l'équilibre de la simulation plutôt fragile.

Au-delà de la modélisation biologique, l'enjeu principal réside dans la programmation concurrente. La simulation repose sur une architecture multi-processus garantissant une isolation totale des agents. Quatre types de processus communiquent en temps réel :

- Le serveur d'environnement (`env.py`) : Il gère l'état global, la repousse des ressources et réagit aux signaux système (SIGUSR1) pour simuler la sécheresse.
- Les agents (`prey.py` et `predator.py`) : Processus autonomes simulant le cycle de vie (faim, reproduction, mort).
- Le tableau de bord (`display.py`) : Interface graphique Pygame permettant de visualiser les populations et d'interagir avec la simulation.

Ce rapport détaille les choix techniques et les mécanismes de synchronisation (Mutex, message queues, sockets) que nous avons choisis pour rendre cet écosystème à la fois stable, réactif et capable de s'arrêter proprement.

2. Architecture, conception technique et IPC

Afin de garantir la cohérence de la simulation, l'état des populations et des ressources est centralisé dans un objet *Manager.dict* (dictionnaire partagé). Cet objet, nommé *shared_state*, stocke en temps réel la quantité d'herbe, le nombre total de proies et de prédateurs. Une variable spécifique, *num_active_preys*, a été ajoutée pour permettre aux prédateurs de savoir combien de proies sont actuellement en mouvement et donc chassables.

Comme plusieurs dizaines de processus peuvent tenter de modifier ces valeurs simultanément, nous avons implémenté un Mutex (`multiprocessing.Lock`). Chaque accès en lecture ou écriture sur la mémoire partagée est systématiquement encapsulé dans un bloc *with lock*. Cela assure l'atomicité des opérations, évitant ainsi les race conditions qui pourraient fausser les compteurs de population lors des naissances ou des décès massifs.

Le processus env fait office de serveur central et de gestionnaire de temps. L'utilisation des sockets TCP (port 1024) sert ici de barrière de synchronisation : au démarrage, chaque agent (prey et predator) doit se connecter au serveur avant de commencer son cycle de vie. Pour que le serveur reste réactif, nous avons utilisé `server.setblocking(False)`, ce qui lui permet de surveiller les nouvelles connexions tout en continuant à gérer la croissance de l'herbe et les signaux système sans interruption.

La communication et les interactions reposent sur deux files de messages distinctes :

- La *msg_queue* sert à remonter les informations vers l'interface graphique (display.py). Les agents y envoient leurs événements (ex: "PRÉDATION !", "Naissance proie") et le serveur y injecte les bilans finaux. C'est aussi par là que passent les commandes utilisateur : la touche **[Q]** envoie un signal d'arrêt (SIGKILL), et la touche **[D]** déclenche une commande de sécheresse forcée (SIGUSR1).
- La *spawn_queue* c'est une nouveauté de notre architecture. Lorsqu'un agent a assez d'énergie pour se reproduire, il dépose un ticket dans cette queue. Le processus principal (main.py) surveille cette file en permanence et génère (spawn) dynamiquement les nouveaux processus agents, garantissant une gestion propre des ressources système.

Enfin, nous avons intégré la gestion des signaux. Le serveur est programmé pour intercepter le signal SIGUSR1. Lorsqu'il le reçoit (déclenchement manuel via l'interface), il déclenche instantanément une phase de sécheresse qui bloque la repousse de l'herbe, impactant ainsi toute la chaîne alimentaire.

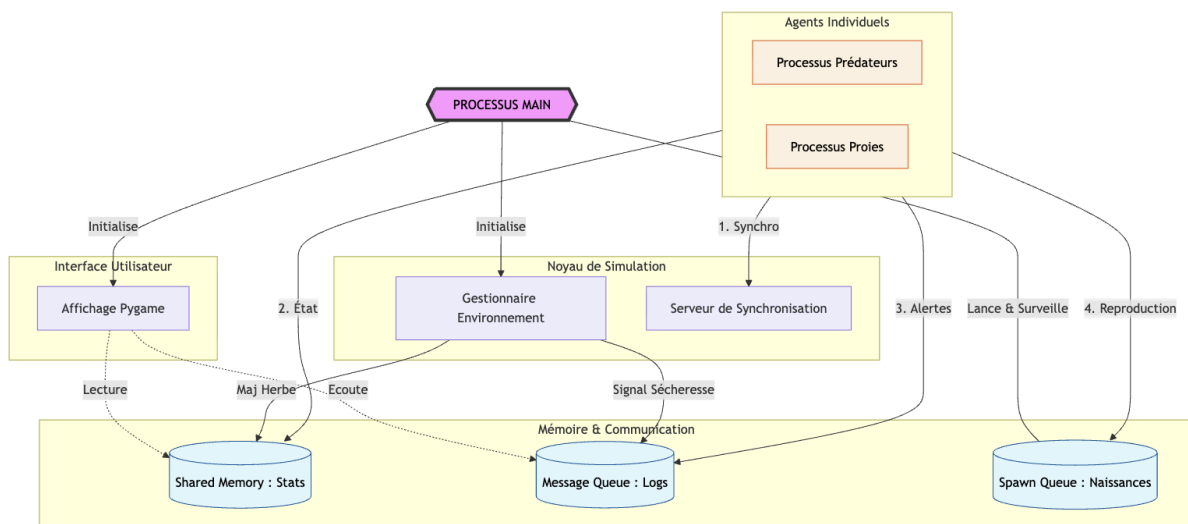


Figure 1: Diagramme de l'architecture du système

3. Modélisation des États

La modélisation par machine à états finis permet de simplifier les comportements des agents en les traduisant en une suite de règles logiques. Chaque processus agent (proie ou prédateur) prend ses décisions de manière autonome en fonction de ses variables internes, ce qui garantit la stabilité et la fluidité de la simulation globale. Chaque individu suit un cycle de vie structuré :

L'agent tente d'abord de se connecter au serveur env via une socket TCP. Cette étape sert de "barrière" pour s'assurer que tous les agents commencent la simulation en même temps.

Une fois connecté, l'agent entre dans une boucle où son énergie décroît cycliquement. C'est ici qu'il varie entre trois sous-états selon ses seuils énergétiques :

- Passif (Repos) : L'agent économise son énergie. Pour la proie, cet état est sécurisant car elle n'est pas chassable par les prédateurs.
- Actif (Faim) : Dès que l'énergie tombe sous un seuil critique (ex: < 50 pour une proie), l'agent passe en mode actif. La proie cherche alors à consommer de l'herbe, tandis que le prédateur tente de chasser les proies signalées comme "actives" dans le dictionnaire partagé.
- Reproduction : Si l'énergie dépasse un seuil de satiété (ex: > 60 pour une proie ou > 160 pour un prédateur), l'agent dépense une partie de ses ressources pour envoyer une demande de naissance via la *spawn_queue*.

Enfin, si la réserve énergétique s'épuise totalement (énergie = 0), l'agent transite vers l'état mort. Il diminue les compteurs de population dans le *shared_state* via le Mutex et ferme son processus proprement. Cette logique de transition permet un équilibre naturel : plus il y a de proies affamées (actives), plus il est facile pour les prédateurs de se nourrir et de se reproduire à leur tour. Cette modélisation est illustrée dans la [*Figure 2*](#).

4. Algorithmes et Mécanismes Système

4.1 Processus Environnement (Serveur)

Le processus env gère la simulation en gérant l'horloge globale et les conditions climatiques. Il intègre un gestionnaire de signaux pour SIGUSR1, permettant de déclencher instantanément un épisode de sécheresse qui suspend la croissance de l'herbe pendant X cycles (à déterminer au début de la simulation). Grâce à l'utilisation de *setblocking(False)*, le serveur maintient une boucle réactive capable de traiter simultanément les nouvelles connexions réseaux (via sockets), les signaux système et la mise à jour de l'état partagé (repousse de l'herbe hors sécheresse). Le pseudo-code de la gestion de l'environnement est présenté en annexe.

4.2 Comportement des Agents (Clients)

Les proies et les prédateurs simulent leur cycle biologique de manière totalement asynchrone. Chaque individu évolue dans son propre processus indépendant et communique ses besoins via la mémoire partagée et les files de messages. Le pseudo-code du cycle de vie des espèces est présenté en annexe.

- La Proie : Elle débute avec une réserve de 50 unités d'énergie et perd 8 unités par cycle (métabolisme). Elle bascule en état "actif" dès que son énergie chute sous le seuil de 50. Dans cet état, elle consomme une unité d'herbe pour regagner 30 unités d'énergie. Si son énergie dépasse 60 unités, elle entre en phase de reproduction : elle dépense 30 unités pour envoyer un ticket à la *spawn_queue*, demandant au processus principal de créer un nouvel agent.
- Le Prédateur : Il suit une logique plus opportuniste. Avec une énergie initiale de 100 unités et une perte de 8 unités par cycle, il entre en phase de chasse dès qu'il tombe sous 120 unités (soit presque immédiatement pour maintenir sa domination). Sa survie dépend exclusivement de la présence de proies actives (celles en recherche de nourriture). Une chasse réussie lui rapporte un gain massif de 80 unités. S'il atteint la satiété au-delà de 160 unités, il se reproduit en dépensant 80 unités via la *spawn_queue*.

Cette spécialisation des seuils permet de stabiliser la simulation et d'éviter une extinction immédiate de l'une des deux espèces.

5. Implémentation et Tests

5.1 Plan de déploiement et validation

La mise en œuvre de notre système a suivi une progression par étapes pour garantir le fonctionnement de chaque composant avant son intégration dans le code global.

Nous avons d'abord testé de manière isolée les algorithmes de survie et les cycles énergétiques des agents (*prey.py* et *predator.py*) pour confirmer les seuils de faim et de reproduction.

Ensuite, on a testé la communication réseau via les sockets TCP et la vérification de l'intégrité du dictionnaire partagé. Nous avons simulé l'accès simultané de nombreux agents pour confirmer que le Mutex (`multiprocessing.Lock`) prévenait efficacement toute corruption de l'état partagé (ressources et compteurs d'activité).

Enfin, nous avons testé la robustesse du processus *main.py* lors de l'utilisation de la *spawn_queue*. Il s'agissait de vérifier que le système pouvait générer dynamiquement des agents sans saturer la mémoire ou bloquer le serveur env.

5.2 Scénarios de défaillance et robustesse

Afin de garantir la stabilité du système, nous avons identifié et traité plusieurs risques critiques. Pour éviter un nombre incohérent d'individus d'une population, un mécanisme de "clamping" ($\max(0, \dots)$) a été appliqué, empêchant les ressources ou les populations d'atteindre des valeurs négatives. De plus, un bloc *finally* est présent dans chaque script d'agent pour s'assurer que, même en cas d'erreur, le processus décrémente sa présence dans le *shared_state* et libère son verrou.

6. Description de l'exécution et utilisation du programme

Pour démarrer la simulation, l'utilisateur doit exécuter le script principal depuis un terminal via la commande `python ./main.py`. Dès le lancement, une phase de configuration interactive s'affiche dans la console. L'utilisateur peut saisir les paramètres suivants (ou utiliser les valeurs par défaut) :

- Quantité d'herbes initiale .
- Population de départ (Proies et Prédateurs).

Une fois la configuration validée, une fenêtre graphique s'ouvre. Elle est divisée en plusieurs sections critiques pour le suivi de l'écosystème.

D'abord un tableau de bord statistique qui affiche en temps réel le stock d'herbe et le décompte précis des populations. Une section spécifique détaille la dynamique des individus. Soit, le nombre de proies en recherche de nourriture (actives) versus celles au repos ou en reproduction et le nombre de prédateurs en chasse et en reproduction. Enfin, une section comptabilise le nombre de naissances et de morts pour chaque espèce. Pendant la simulation, on voit également l'état climatique avec "Alerte Sécheresse".

L'opérateur dispose de deux commandes directes pour influencer ou stopper la simulation :

- Touche [D] : Force manuellement le déclenchement d'une sécheresse. Techniquement, cela envoie un signal SIGUSR1 au processus `env`, interrompant immédiatement la repousse des ressources.
- Touche [Q] : Demande un arrêt propre. Une commande "STOP" est envoyée dans la file de messages, permettant au superviseur de fermer chaque processus agent sans laisser de processus zombies.

La simulation s'arrête également de manière automatique si l'une des deux espèces disparaît totalement. Un écran de Bilan Final s'affiche alors, récapitulant les l'état des populations et ressources au moment de l'arrêt et il y a un graphique de population dynamique permettant de visualiser l'évolution des trois courbes (Herbe, Proies, Prédateurs) sur la durée, facilitant l'observation des cycles de vie.

7. Conclusion

Ce projet nous a permis de mettre en pratique les connaissances fondamentales acquises en TD de PPC : la gestion de la mémoire partagée via un dictionnaire centralisé, la synchronisation rigoureuse par verrous (Mutex) et la communication inter-processus par Sockets et Files de messages. L'utilisation d'une machine à états finis pour chaque agent a été déterminante pour transformer des comportements biologiques (faim, reproduction, prédation) en un code structuré, capable de s'exécuter de manière totalement asynchrone.

Cependant, la réalisation de ce système multi-processus a révélé des défis techniques inattendus. Le point le plus complexe a été la gestion des signaux système (SIGUSR1) et des entrées clavier. Nous avons observé que lorsque la population d'agents devient trop importante, le système d'exploitation sature, ce qui entraîne une latence dans l'interception des commandes [D] et [Q]. Cela montre les limites de l'approche "un processus par individu" sur une machine standard et souligne l'importance d'optimiser les temps de pause (sleep) pour laisser respirer le scheduler.

Aussi, maintenir une interface graphique fluide sous Pygame tout en gérant une file de messages saturée par les naissances et les morts a nécessité un réglage de la fréquence de rafraîchissement. Nous avons dû veiller à ce que les phases de verrouillage du `shared_state` soient les plus courtes possibles pour ne pas bloquer les processus vitaux de la simulation.

En conclusion, "The Circle of Life" est une démonstration concrète de la puissance et de la complexité du parallélisme. Ça serait intéressant de passer à une architecture hybride (mélangeant threads et processus) ou d'implémenter une gestion spatiale (grille 2D) pour limiter les interactions des prédateurs aux proies les plus proches, réduisant ainsi les calculs globaux.

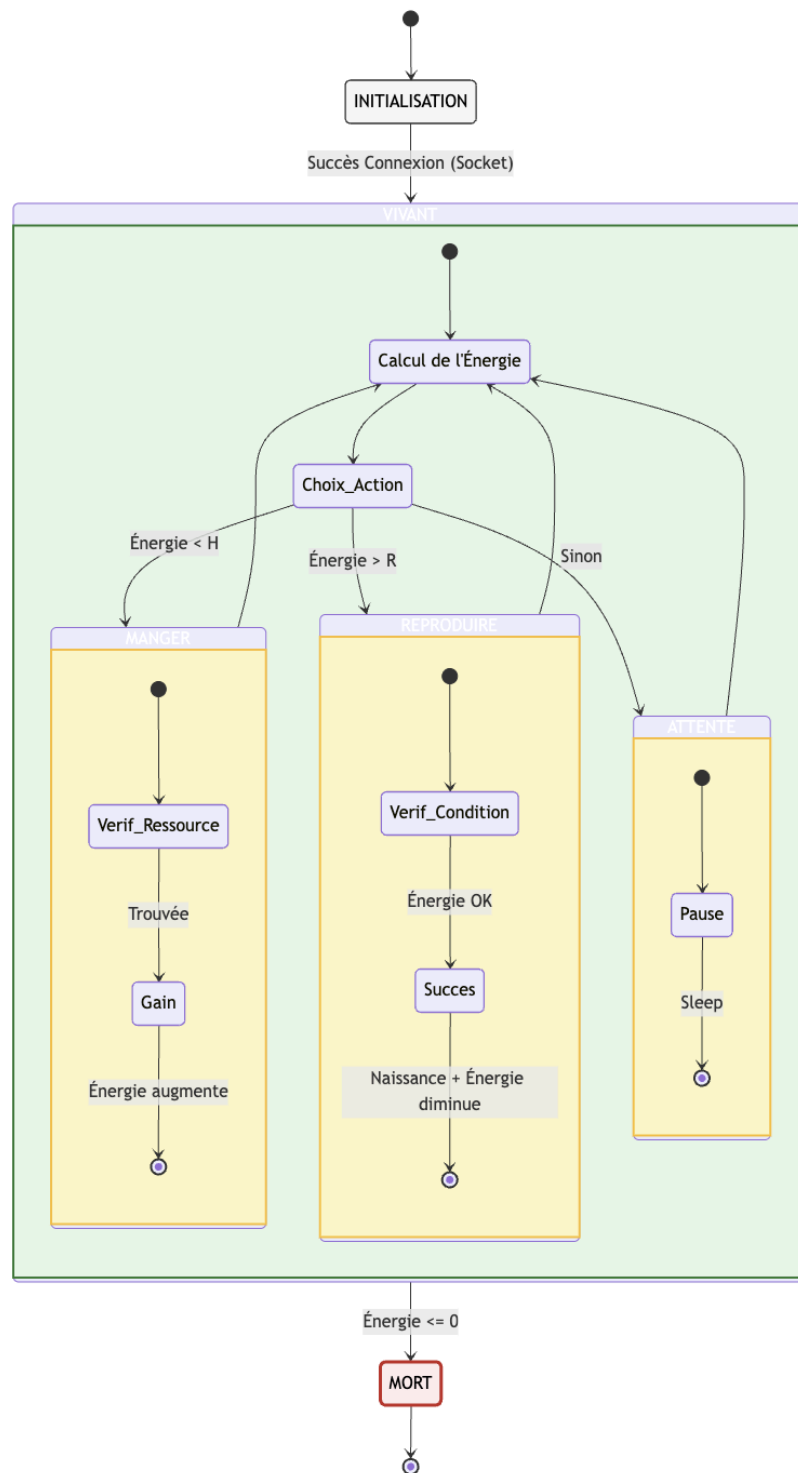


Figure 2: Schéma de la machine à états

ANNEXE: Utilisation de l'IA

Cette annexe détaille la manière dont l'IA (Gemini, ChatGPT) a été utilisée comme outil d'accompagnement technique. Dès le début du projet, l'IA nous a aidés à structurer notre architecture multi-processus. Nous avons utilisé le modèle pour valider la pertinence de séparer la logique métier (agents) de la logique d'affichage (Pygame). Cette réflexion nous a conduits à implémenter deux files de messages distinctes : la `msg_queue` pour l'affichage et la `spawn_queue` pour la gestion dynamique des naissances par le processus parent, évitant ainsi de surcharger le serveur d'environnement.

L'IA a été particulièrement utile pour traiter des mécanismes système complexes. Elle nous a guidés sur l'utilisation de `setblocking(False)` pour le serveur socket, nous permettant de surveiller les signaux système (SIGUSR1) et les timers de repousse de l'herbe tout en écoutant les nouvelles connexions.

Nous avons utilisé l'IA pour comprendre comment intercepter proprement un signal afin de modifier l'état d'une variable globale (sécheresse) sans interrompre la boucle principale du serveur. L'IA nous a aussi aidé à identifier les risques de *race conditions* lors de la mise à jour simultanée des compteurs de proies actives. Cela nous a conduits à systématiser l'usage du Mutex (`multiprocessing.Lock`) autour du dictionnaire partagé.

N'ayant aucune connaissance préalable de la bibliothèque Pygame, nous avons massivement sollicité l'IA pour concevoir toute la partie `display.py`. Cette collaboration a été essentielle pour traduire nos données brutes en une interface visuelle cohérente (graphiques dynamiques, logs, compteurs d'activité).

L'usage de l'IA n'a jamais remplacé notre propre analyse. Chaque suggestion de code a été testée et adaptée manuellement, comme l'intégration du mécanisme de "clamping" et l'adaptation des seuils pour que la simulation corresponde intégralement à nos attentes. Chaque point où nous avons utilisé l'ia a été suivi d'une compréhension de notre part afin de pouvoir manipuler au mieux le rendu final de ce projet.

ANNEXE: Pseudo-code des algorithmes importants

ALGORITHME Cycle_de_vie_Agent

DEBUT

TENTER connexion Socket vers "localhost:1024"

SI échec ALORS

QUITTER FERMER Socket

TANT QUE (Energie > 0) FAIRE

SI (Type est PROIE) ALORS

SI (Energie < 50 ET NON active) ALORS

VERROUILLER(Mutex)

active <- VRAI

Proies_Actives <- Proies_Actives + 1

DEVERROUILLER(Mutex)

SINON SI (Energie >= 50 ET est_active) ALORS

VERROUILLER(Mutex)

active <- FAUX

Proies_Actives <- MAX(0, Proies_Actives - 1)

DEVERROUILLER(Mutex)

FIN SI

FIN SI

Energie <- Energie - 8

SI (Energie < H) ALORS

VERROUILLER(Mutex)

SI (Type est PROIE ET Herbe > 0) ALORS

Herbe <- Herbe - 1

Energie <- Energie + 30

ENVOYER "Manger" vers Message_Queue

SINON SI (Type est PREDATEUR ET Proies_Actives > 0) ALORS

Population_Proies <- MAX(0, Population_Proies - 1)

Proies_Actives <- MAX(0, Proies_Actives - 1)

Energie <- Energie + 80

ENVOYER "Prédation" vers Message_Queue

FIN SI

DEVERROUILLER(Mutex)

SINON SI (Energie > R) ALORS

Energie <- Energie - Coût_Repro

ENVOYER Type_Agent vers Spawn_Queue

ENVOYER "Naissance" vers Message_Queue

FIN SI

ATTENDRE 0.8 à 1.0 seconde

FIN TANT QUE

VERROUILLER(Mutex)

Population_Totale_Type <- MAX(0, Population_Totale_Type - 1)

SI active ALORS

Proies_Actives <- MAX(0, Proies_Actives - 1)

DEVERROUILLER(Mutex)

ENVOYER "Mort" vers Message_Queue

FIN

ALGORITHME Gestion_Environnement

DEBUT

INITIALISER Socket Serveur en mode NON-BLOQUANT

DEFINIR handle_SIGUSR1 :

Sécheresse_Active <- VRAI

Timer_Sécheresse <- 8

TANT QUE (Simulation_Active) FAIRE

SI Message_Queue contient ("STOP") ALORS

Simulation_Active <- FAUX

SI Message_Queue contient ("FORCED_DROUGHT") ALORS

ENVOYER SIGUSR1 à soi-même

VERROUILLER(Mutex)

SI (NON Sécheresse_Active) ALORS

Herbe <- Herbe + 35

SINON

Timer_Sécheresse <- Timer_Sécheresse - 1

```
SI (Timer_Sécheresse <= 0) ALORS  
  Sécheresse_Active <- FAUX
```

```
FIN SI
```

```
Herbe <- MAX(0, Herbe)  
DEVERROUILLER(Mutex)
```

```
FIN SI
```

```
TENTER Acceptation_Connexion_Socket  
SI nouvelle_connexion ALORS  
  FERMER(nouvelle_connexion)  
ATTENDRE 0.1 seconde
```

```
FIN TANT QUE  
FIN
```